Project 3 Design Document

Graded

23 Hours, 21 Minutes Late

Group

Owen Ye
Stella Song
Lucas Johansson
...and 1 more
View or edit group

Total Points

12.75 / 14.5 pts

Question 1

Buffer Cache — **4** / 4.5 pts

1.1 Data Structures and Functions — **1** / 1 pt

- ✔ **+ 0.25 pts** Allocates space for 64 cache entries either statically or dynamically.
- ✔ **+ 0.25 pts** Outlines cache read and write methods that takes in a sector number of type `block_sector_t` and buffer of type `void *`.
- ✔ **+ 0.25 pts** Each cache entry contains a buffer of size `BLOCK_SECTOR_SIZE` and then sector number.
- ✔ **+ 0.25 pts** Each cache entry contains a dirty bit.

+ 0 pts Blank / Incorrect

1.2 Algorithms — **1** / 1 pt

- ✔ **+ 0.25 pts** Initializes all cache entries properly (i.e. invalid) on system startup.
- ✔ **+ 0.5 pts** Evicts entries using LRU or a valid approximation to LRU (e.g. clock).
- ✔ **+ 0.25 pts** Flushes all valid dirty cache entries on system shutdown.

+ 0 pts Blank, majorly incorrect, or lacks substantial detail.

1.3 Synchronization — **1.75** / 2.25 pts

- ✔ **+ 0.25 pts** Serializes concurrent reads and writes to the same block.
- ✔ **+ 0.5 pts** Concurrent reads and writes to different blocks are not serialized (i.e. does not block while holding the global lock).

+ 0.5 pts Evicts and loads new blocks into the cache in a race-free way (e.g. handles multiple concurrent evictions which could result from context switching in between releasing the global cache lock and acquiring the entry-specific lock).

- ✔ **+ 0.5 pts** Searches for an entry in the cache in a race-free way (e.g. acquiring a global lock).
- ✔ **+ 0.25 pts** Updates metadata of cache entries in a race-free way (e.g. acquiring a global lock).
- ✔ **+ 0.25 pts** Removes global file system lock.

+ 0 pts Incorrect / Blank

1.4 Rationale — **0.25** / 0.25 pts

- ✔ **+ 0.25 pts** Overall, each rationale is properly justified and conveys why each algorithm was chosen.

+ 0 pts Blank, majorly incorrect, or lacks substantial detail.

Question 2

Extensible Files **3.25** / 4.5 pts

2.1 Data Structures and Functions **1** / 1 pt

✔ **+ 0.25 pts** Removes `struct inode_disk` completely from struct inode. If storing a pointer to `struct inode_disk`, needs to handle cache evictions properly.

✔ **+ 0.5 pts** Adds at least one double indirect pointer to support a maximum file sizes of 8 MiB.

✔ **+ 0.25 pts** Calculates appropriate amount of unused space such that `struct inode_disk` is of size `BLOCK_SECTOR_SIZE` bytes.

+ 0 pts Blank, majorly incorrect, or lacks substantial detail.

2.2 Algorithms **1.25** / 1.5 pts

✔ **+ 0.5 pts** Traverses through the tree of pointers to find the correct block for `inode_read_at` and `inode_write_at`.

✔ **+ 0.5 pts** Grows files by resizing inodes (e.g. using a inode_resize method similar to one shown in discussion).

+ 0.25 pts Zeroes out intermediate sectors that get allocated as a result of extending a file. If not implementing "sparse files", these sectors get allocated (and zeroed out) at the time of extension. If implementing "sparse files", the intermediate sector can get allocated at a future read/write, but the sector must be zero-filled nonetheless.

✔ **+ 0.25 pts** Rolls back all block allocations and resets pointers on allocation failures.

+ 0 pts Blank, majorly incorrect, or lacks substantial detail.

2.3 Synchronization — 0.75 / 1.75 pts

+ 0.5 pts Resolves concurrent resizes in a race-free way. This can be achieved by relying on the buffer cache's sector-level synchronization since the inode itself is stored in a single sector. If this approach is taken, one needs to ensure that the per-entry lock for the inode's sector's cache entry is held throughout the entirety of the inode resizing function.

An alternate approach is to just use a new lock that is associated with the `struct inode`.

+ 0.5 pts Resolves concurrent read and a resize in a race-free way.
The inode's `length` metadata should not get changed inside the resizing function, since it is possible that the call to `resize` comes from a call to `inode_write_at`, where a file is being extended. If the writing thread extends the file and updates `length` and then we context switch to a reader thread, the reader can read data that has not been written to yet (the updated `length` field will allow this).

+ 0 pts Synchronizes count for denied writes and handles case where one thread calls file_deny_write concurrently with another thread's write. [Rubric item is worth 0 points so as to note double penalize students for missing item 5.]

✔ **+ 0.25 pts** Synchronizes the free map explicitly with a lock or indirectly through the buffer cache (i.e. cache the free map).

✔ **+ 0.25 pts** Synchronizes access to `struct inode` metadata fields with a lock.

✔ **+ 0.25 pts** Synchronizes access to `struct list open_inodes` with a lock.

+ 0 pts Blank, majorly incorrect, or lacks substantial detail.

2.4 Rationale — 0.25 / 0.25 pts

✔ **+ 0.25 pts** Overall, each rationale is properly justified and conveys why each algorithm was chosen.

+ 0 pts Blank, majorly incorrect, or lacks substantial detail.

Question 3

Subdirectories **3.5** / 3.5 pts

3.1 Data Structures and Functions **1** / 1 pt

✔ **+ 0.25 pts** Keeps track of the CWD of type `struct dir *` for each process in `struct process`.

✔ **+ 0.25 pts** Differentiates if an inode corresponds to a directory or file with either a field in `struct inode_disk` or the corresponding directory entry in the parent directory.

✔ **+ 0.5 pts** Adds support for directories in the file descriptor table.

+ 0 pts Blank, majorly incorrect, or lacks substantial detail.

3.2 Algorithms **1.75** / 1.75 pts

✔ **+ 0.5 pts** Resolves both absolute and relative paths by traversing through directories using `dir_lookup`.

+ 0 pts Automatically add entries for `"."` and `".."` when creating a directory (i.e. doesn't handle them in a special case).

✔ **+ 0.5 pts** Declares whether the CWD of a process or open directories can be deleted. If allowed, explains how to recover when a CWD is deleted (e.g. update `removed` attribute and check on every access). If disallowed, explains the mechanism behind disallowing deletion (e.g. check the `open_cnt` member in `struct inode`).

✔ **+ 0.25 pts** Child processes inherit the CWD from parent process.

✔ **+ 0.25 pts** Explicitly disallow the `read` and `write` syscalls on directories, since these should be done via the `readdir` and `mkdir` syscalls instead.

✔ **+ 0.25 pts** Updates existing syscalls to use relative or absolute paths and mentions the implementation of the new system calls.

+ 0 pts Blank, majorly incorrect, or lacks substantial detail.

3.3 Synchronization **0.5** / 0.5 pts

✔ **+ 0.5 pts** Includes a lock for synchronizing directory operations (e.g. reusing the one from Task 2).

+ 0 pts Blank, majorly incorrect, or lacks substantial detail.

3.4 Rationale **0.25** / 0.25 pts

✔ **+ 0.25 pts** Overall, each rationale is properly justified and conveys why each algorithm was chosen.

+ 0 pts Blank, majorly incorrect, or lacks substantial detail.

Question 4

Concept Check & Design

2 / 2 pts

Write-Behind

- ✔ **+ 1 pt** Correct: Create kernel thread that flushes the cache periodically.
- ✔ **+ 0.5 pts** Partial: initiates flushing the buffer cache in interrupt context. Recall, I/O should not be done in an interrupt context.
- ✔ **+ 0.25 pts** Partial: flushes dirty nodes on each cache operation. Recall, we want this to be done in the background.

Read-Ahead

- ✔ **+ 0.5 pts** Prefetches blocks using a reasonable policy (e.g. linear scans or prefetching blocks from an inode).
- ✔ **+ 0.5 pts** Uses background thread so the prefetching doesn't block the thread performing the original operation (otherwise there's no speedup, since it is blocked anyway).

+ 0 pts Blank, majorly incorrect, or lacks substantial detail.

Question assigned to the following page: 1.1

Project File System Design

Group 75

Name	Autograder Login	Email
Owen Ye	student218	owen2608@berkeley.edu
Stella Song	student263	hyeseung1116@berkeley.edu
Merrick Zheng	student246	zhengmerrick@berkeley.edu
Lucas Johansson	student349	lucasjohansson@berkeley.edu

Buffer Cache

Data Structures and Functions

```
struct buffer_cache_entry {
  block_sector_t  sector;
  bool            valid;
  bool            dirty;
  bool            accessed;
  bool            is_meta;
  bool            meta_skipped
  int             pin_count;
  struct lock     lck;
  struct condition  cv;
  uint8_t         data[BLOCK_SECTOR_SIZE];
};

struct buffer_cache {
  struct buffer_cache_entry  entries[CACHE_SIZE];
  int            clock_hand;
```

Question assigned to the following page: 1.2

```
  struct lock    table_lock;
};
```

Algorithms

Initialization & Shutdown

1. buffer_cache_init() called within filesys_init()
   a. Allocate and zero out a global buffer_cache
   b. For all entries initialize their values and lock
2. buffer_cache_shutdown() called within filesys_done()
   a. Acquire cache.table_lock and for each entry acquire its lock and check whether the entry is dirty and valid. If so call block_write()
   b. Release all locks

Clock Algorithm

Case of cache hit: Lookup & Pinning (return a locked, pinned buffer entry containing sector)

- Lookup under cache.table_lock
- Loop scan through the entries to find an entry that is valid and matching sector
- Acquire its entry lock
- Increment its pin_count, set access to true, release table_lock
- If the entry is not valid (another thread is still loading) we can cond_wait on the entry lock.

Case of cache miss:

- Under table_lock we find the entry to evict which we will call victim (to find victim go to section below).
- Acquire the victims lock.
- If the entry is dirty, call block_write()
- Reset the victims entry such that a new sector was initialized. ie setting pin count to 0 and so on
- Finally release the table_lock

Load Block Data:

- Under victim→lock

```
block_read();
victim->valid = true;
```

Questions assigned to the following page: 1.4 and 1.3

```
cond_broadcast(&victim->lock);
```

- Return the locked entry

Finding the victim:

1. Loop indefinetly
2. Use the clock hand as the index for entries, and on each iteration increment by 1 mod # of entries.
3. For each iteration do the following:
   a. Acquire the lock for the entry
   b. If pin count > 0 release lock and continue
   c. If entry is accessed, set accessed to false, release lock, and continue
   d. If entry is_meta and !meta_skipped. We know we haven't given our meta data a second chance, so we set meta_skipped to true, release lock, and continue.
   e. If no condition is met, our victim is the current block and should be returned.

Replace uses of block_read and block_write to utilize caches to retrieve buffer

1. Get buffer as described above
2. call memcpy()
3. Set the buffer access to true
4. Decrement its pin count
5. If it was a write set dirty bit to true
6. Release the lock on the buffer

Synchronization

We utilize a global lock for buffer caches called table_lock. This is so only one thread can insert, remove or pick a victim entry. Furthermore, each buffer entry has a lock. This is to guard its mutable fields such and having a lock for each entry, guarding its mutable fields like pin_count, valid, dirt, etc. Within each entry we also have a condition variable that allows any thread that finds valid == false (another thread is loading the block) sleep until the loader finishes. Once block_read() completes, the loader sets valid to true and broadcasts to wake all waiters. The pin_count also prevents the eviction of any block in active use.

Rationale

We utilize a clock algorithm with enhancements to improve file system performance. Regarding time complexity, our design works at O(1) for a cache hit because of the fixed 64-block limit of the cache blocks. We are using the clock algorithm over other

Questions assigned to the following page: 1.4 and 2.1

alternatives as it has good hit rate with lower overhead. This is because the need for expensive list manipulations is not needed. Two-level locking system allows for more efficient concurrency while also preventing race conditions for more important operations. The victim policy's second chance policy allows us to more fairly decide which blocks to evict, as more important blocks can have another look. This is mainly done because metadata blocks are used more often and are extra important. This is seen in the is_meta field within the struct. So, we don't want to evict these unintentionally, and the second chance approach prevents this. This approach is more effective than randomly choosing blocks to remove. Pinning is used to keep used blocks safe. This prevents them from being evicted. Overall, this process allows us to more efficiently work with the cache, freeing blocks that aren't used and preventing freeing of useful blocks, while being algorithmically efficient.

Extensible Files

Data Structures and Functions

```
struct inode_disk {
...
  unsigned magic
  block_sector_t    direct_blocks[12];
  block_sector_t    indirect_block;
  block_sector_t    double_indirect_block;
  uint32_t.         unused[111]
...
};

struct inode {
...
  struct rw_lock inode_rw_lock
...
}
```

Question assigned to the following page: 2.2

Algorithms

inode_write_at:
if byte_to_sector() returned -1, we haven't allocated the data block corresponding to that index. First calculate the sector number with offset/BLOCK_SECTOR_SIZE. If within the first 12 sectors allocate and zero new direct block. If in the next 128 sectors, if the indirect block has not yet been allocated, allocate and zero it. then allocate the data block, and put a reference to the datablock in the correct location in the indirect block (sector number - 12). If greater than the indirect block range, allocate a new zeroed double indirect block if one has been allocated yet. If the indirect block has been allocated for (sector_index - 140)/128, allocate one if needed, and put the reference in the double indirect block. Allocate the data block at (sector_index - 140)%128, and put the reference in the indirect block at the correct position. We use free_map_allocate() to get a free block and block_write() to zero them. During this process of allocating new blocks, we maintain a list of blocks that have been allocated so far. If the disk runs out of space, we can then release all blocks in this list by modifying the free_map. Reset the inode length.

inode_read_at:
change to read from cache first, and only go to disk if data is not found in the buffer cache. If the sector we want to access is not allocated, but we are within the file length, return a buffer of zeroes.

byte_to_sector:
sector index is offset / BLOCK_SECTOR_SIZE
if sector index < 12 , return data.directed_blocks[sector index]
else if sector index < 12 + 128
return block at indirect block sector index - 12
else if sector < 12 + 128 + 128*128
block read at double_indirect_block. Then block read at the (sector index - 140)/128 entry of the block gotten back. return the data block address of the entry at [(sector index - 140)%128] in the gotten block.

Bytes to sector should used the buffered cache when possible, and only go to disk when entry is not in cache. If anything not allocated, return -1

inumber

Questions assigned to the following page: 2.2, 2.4, and 2.3

Call inode_get_inumber for the corresponding file, gotten from the fd table.

Sparse files:
To implement sparse files, we follow the same regular process of inode_write_at, and only allocate the data block corresponding to the write, with blocks from the previous EOF to the new block remaining empty. However, we do update the length of inode to include the unallocated blocks in the middle. On a read, if we want to access an offset that is within the length but it is in an unallocated region, we just return 0s.

Synchronization

Synchronize free_map access with a global free_map_lock, in order to prevent race conditions from when we have concurrent calls to allocate and deallocate data blocks. In struct inode, include a inode_rw_lock which is a rw_lock, which we acquire in inode operations such as inode_write_at and inode_read at with reader specified to the correct value. This is because multiple processes could be referring to the same inode, and we wan't to prevent concurrency issues when writing to the same inode, but at the same time, can allow concurrent reads, but only exclusive write access.

Rationale

We changed the inode_disk struct to include 12 direct access for quick access to small files, a indirect_block for slightly bigger files, and double_indirect_block to support big files. This is to handle both small and large files efficiently, as well as to support appending to files to scale file size dynamically, without fragmentation. We changed inode write and inode read to first first hit the cache/write to the cache because we want to avoid slow disk operations as much as possible. For the inode struct, because each inode is a sector (512 bytes), we need to have an array of 111 4 byte values in order to fill the space. We chose to use a rw_lock because we need to have exclusive access to the inode on writes, but for reads, we can have concurrent readers. This should be more efficient than just using a regular lock. For our recovery mechanism, we need to ensure that we rollback to a good state, so we store the data that we changed and the new data blocks, so we can deallocate them if the operation fails.

Subdirectories

Questions assigned to the following page: 3.1 and 3.2

Data Structures and Functions

```
struct inode_disk {
...
    bool is_dir;
...
};

struct dir {
...
  struct lock lck;
...
};

struct process {
...
    struct dir* cwd;        // Current working directory
    char cwd_name[PATH_MAX]; // Absolute path of cwd
...
};
```

Algorithms

Path Resolution:
Every syscall that takes a filesystem path first calls resolve_path, a helper function that takes a string like “/a/b/../c” into a directory object.

resolve_path(path)

1. Decide the starting point. If the path begins with ‘/’ start at the root directory, otherwise start at the process’s current working directory, stored within the PCB.
2. Split the components using get_next_part.
3. Handle each component case by case except last component
    a. If the component is “.”, ignore it and stay int he same directory

Question assigned to the following page: 3.2

   b. If the component is ".." look up the parent entry from the current directory and switch to that directory.
   c. Otherwise, look up the component name in the current directory. If it is not found, report an invalid path, otherwise move into that directory.
   d. Note: Done using dir_lookup()
4. Handle final component.
   a. If it is a name, return (current_dir, name)
   b. If it ended in a slash return (current_dir, empty-name)

Setting CWD Per-Process

- On startup, set the cwd to the root directory.
- On fork/exec, deep-copy (dir_reopen) the parent's cwd so that the child inherits it.
- On exit, close the cwd

Syscalls

1. chdir
   a. Call resolve_path
   b. Close the old cwd, and set it to the resolved directory and return true.
   c. On any errors return false.
2. mkdir
   a. Call resolve_path to resolve to (parent_dir, new_name)
   b. If the resolution fails or new_name is empty, return false.
   c. Lock the parent's directory.
   d. If new_name exists in parent_dir, or is "."/".." unlock and fail.
   e. Allocate a new inode sector marked as a directory.
   f. In the new directory, create entries "." pointing to itself and ".." pointing to the parent's inode number.
   g. Insert new_name → new inode into parent_dir.
   h. Unlock and return whether the insert succeeded.
3. readdir
   a. Verify that the FD refers to a directory
   b. Maintain a read-position inside the directory object
   c. Repeatedly read the next on-disk directory entry record at that position
   d. If the entry is in use and its name is not "." and ".." copy its name into the user buffer and return true
   e. Advance the position pointer by one record and repeat until the end of the file, then return false.
4. isdir

Questions assigned to the following page: 3.3, 3.4, and 3.2

a. Look up the FD's underlying inode and return whether its "is-directory" flag is set.

5. inumber
   a. Return the on-disk sector number of the inode backing the given FD. If invalid return -1.

Updates to open/close/exec/remove

1. Open
   a. resolve_path to (dir, name)
   b. If name is empty, open the directory itself, otherwise lookup name in dir
   c. If found and is a file → open as a file FD. If it is a directory open as a dir FD.
2. Close
   a. If FD is for a directory, close the directory, otherwise close the file.
3. Exec
   a. After creating the new process but before it runs, copy the parent's current working directory to the child.
4. Remove
   a. Resolve the path (parent_dir, target_name)
   b. Lookup the target_name in parent_dir. If it doesn't exist, fail.
   c. If it names a directory, open it and scan its entries. If it contains anything besides "."/".."., then fail
   d. Remove the directory entry from parent_dir and free all blocks of the inode
   e. If it names a file, do the analogous removal.

Synchronization

We are using a lock within the dir for mkdir. When we create a new file specifically so that multiple processes cannot create the same directory at the same time. We decided on this lock to be at the directory level instead of globally so that we are only locking the relevant directory and not all directories concurrently, keeping the critical section as tight as possible.

Rationale

We implement a lock struct within the dir struct so that whenever we call mkdir, we can incorporate synchronization in multiple processes making the same directory at the same time. Within our struct process, we implemented the attributes of the current directory of this process as well as the full command path. In terms of our helper method, resolve_path, this was a crucial method that we needed to

Question assigned to the following page: 4

implement as it simplifies path traversal and resolution logic across many of the system calls that manage the file paths. This method handles both of the relative and absolute paths and helps with the logic of simplifying fragmented file paths. In our updated remove function call, we decided on allowing deletion of a directory that is open by a process or in use as a process's current working directory. Thus, we ensure that we do not allow the opening or creation of files in a deleted directory as our helper method resolve_path checks for this.

Concept check

1. The write behind can be implemented initializing a thread that periodically scans through the buffer cache, and for dirty sectors, writing them to disk, then sleeps for an allotted amount of time with timer_sleep. We can then set the dirty bit in the buffer cache to not be dirty.
2. Create a dedicated read ahead thread that reads from a read ahead queue. Whenever we want to fetch a block n, we add block n + 1 to the read ahead queue. The read ahead thread asynchronously fetches blocks in the read ahead queue and loads it into the buffer cache.